

F R O M

T O

ALERT → **ANSWER**

Accelerating Anomaly Investigation with OpenSearch Agent Skills

The detector finds the signal. This talk is about what happens next.

THE PROBLEM

Detection lag is solved.
Investigation lag isn't.

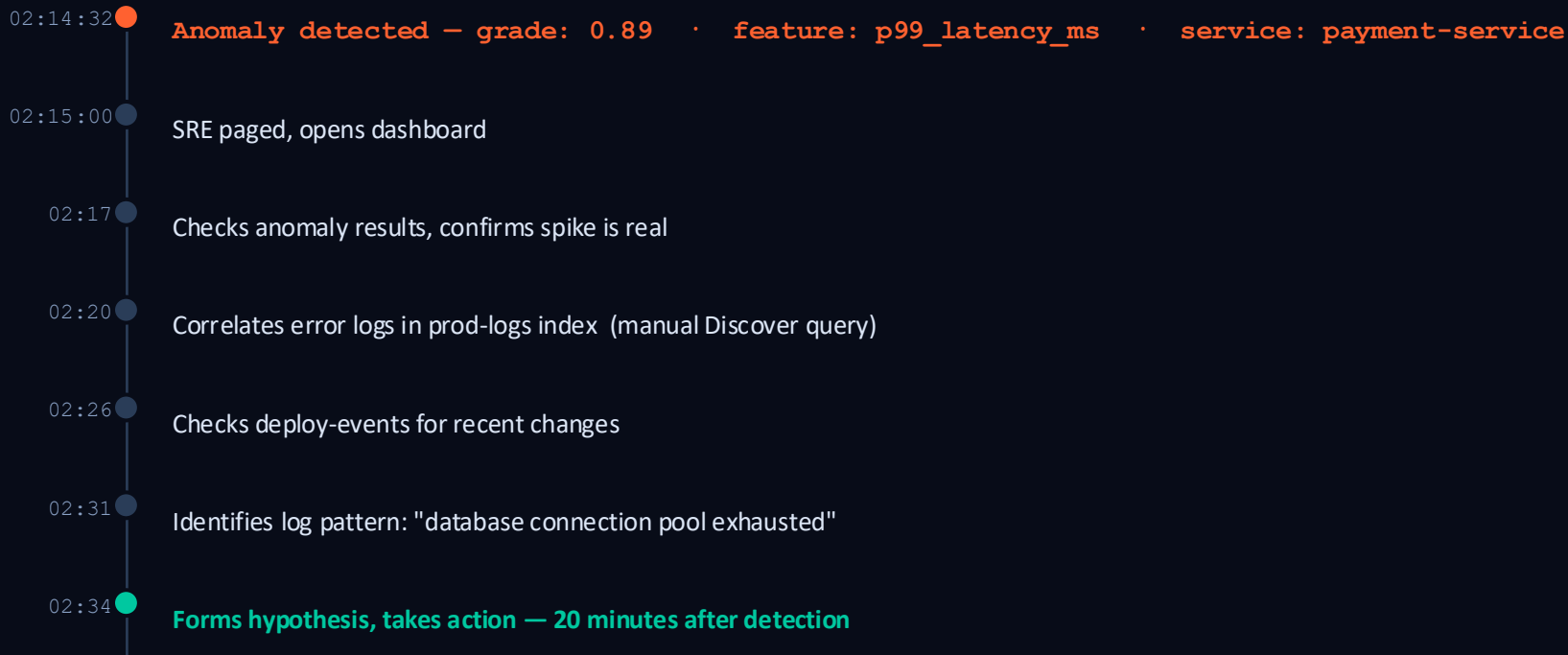
~20 min

average time from alert to hypothesis
for an on-call SRE

The anomaly detector fires in milliseconds. The SRE then spends 20 minutes correlating logs, cross-referencing metrics, and scanning deploy history before knowing where to look.

That gap is the problem.

What happens between the alert and the action.



WHAT YOU HAVE

OpenSearch knows there's a problem.

WHAT THE DETECTOR RETURNS

✓ Anomaly score: 0.89

✓ Timestamp: 02:14:32 UTC

✓ Feature: p99_latency_ms

✓ Confidence: HIGH

✓ Grade threshold: > 0.7

WHAT IT DOESN'T TELL YOU

– Root cause

– Which component failed

– What changed recently

– Correlated error patterns

– Recommended action

THE SOLUTION

Connect detection to investigation.



TOOLS IN THE FLOW

`SearchAnomalyResultsTool`

confirm the anomaly is real

`SearchIndexTool` (prod-logs)

correlate error log spikes

`SearchIndexTool` (deploy-events)

identify recent deployments

`LogPatternTool`

cluster repeated log patterns

`MLModelTool`

synthesise ranked hypotheses from tool output

No humans in the loop until the report is ready!

Useful agents ask small, bounded questions.

01 **Deterministic before generative**

`Search tools run first. LLM synthesises last. Never let the model search.`

02 **One question per tool**

`Each tool answers exactly one thing. Overlap creates hallucinated confidence.`

03 **Fail fast and transparently**

`An empty result is information. A hallucinated one is a pager storm waiting to happen.`

04 **Time-bound every query to the anomaly window**

`now-10m is not a guess — it's the anomaly detection window. Use it or get noise.`

02:14 AM. Agent triggered.



02:14:32 **ALERT** anomaly grade 0.89 · feature p99_latency_ms · service payment-service

02:14:33 **AGENT** flow agent triggered via webhook

02:14:33 ▶ SearchAnomalyResultsTool querying prod-metrics ... 47ms

02:14:33 ▶ SearchIndexTool (prod-logs) now-10m, ERROR/WARN ... 112ms

02:14:33 ▶ SearchIndexTool (deploys) now-60m ... 89ms

02:14:34 ▶ LogPatternTool clustering 847 log lines ... 203ms

02:14:34 ▶ MLModelTool synthesising ... 1.2s

02:14:35 **REPORT** **HIGH CONFIDENCE** – deploy-induced regression

payment-service v2.4.1 → v2.4.2 deployed 02:08 UTC

pattern: "database connection pool exhausted" (62% of errors)

recommendation: rollback or increase pool size (max_conn: 20 → 100)

The things that will humble you.

LLM synthesis is the weakest link

Tools hallucinate less than models do. Every token the model adds is a liability.

Wide tool access → confident wrong answers

More data is not always more signal. Unrelated indices produce false correlations.

Alert fatigue transfers to agent fatigue

If the anomaly detector fires constantly, so does the agent. Tune detection first.

Index naming can fail silently

prod-logs ≠ production-logs — the agent doesn't know and won't warn you.

Time windows drift

now-10m is wrong when the anomaly happened 45 minutes ago. Match window to detector interval.

Where human judgment still wins.

SCENARIO

WHY THE AGENT FALLS SHORT

Novel failure modes

The agent has never seen this pattern. Pattern matching fails on first-time failures.

Blast radius decisions

"Is it safe to rollback right now?" requires knowing what else is deployed.

Rollback authorisation

Someone still has to say yes. The agent surfaces the hypothesis; the human owns the action.

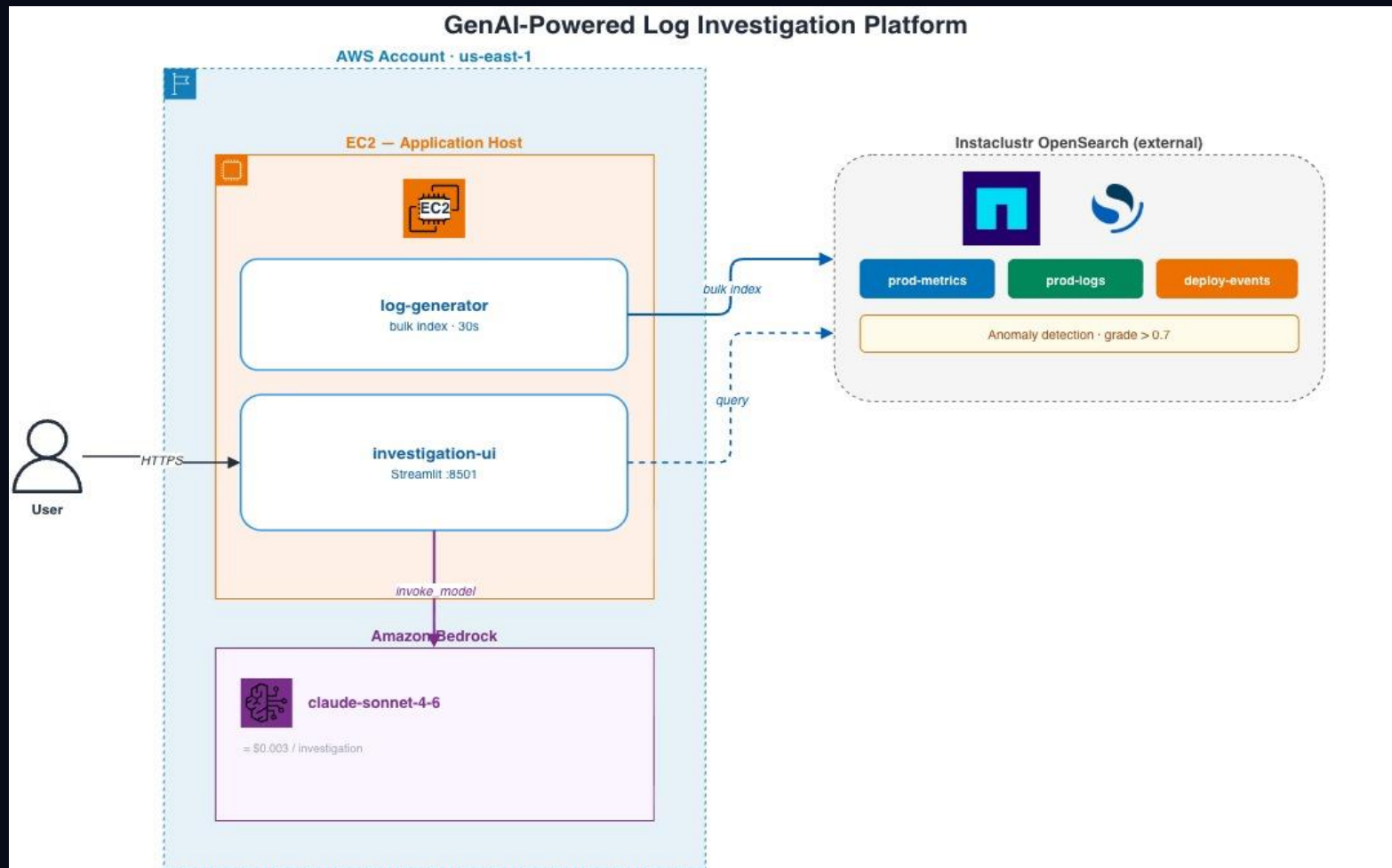
Cross-team coordination

Slack, stakeholder communication, and escalation are not agent output.

Context the logs don't hold

"We pushed this exact fix last quarter." That knowledge lives in people, not indices.

Architecture



DEMO TIME!

Three layers. One wire.

```
anomaly_detector
├── grade > 0.7
│   └── alerting_monitor
│       ├── fires webhook
│       └── flow_agent
│           ├── runs tools
│           │   ├── SearchAnomalyResultsTool
│           │   ├── SearchIndexTool (prod-logs)
│           │   ├── SearchIndexTool (deploy-events)
│           │   ├── LogPatternTool
│           │   └── MLModelTool
│           └── synthesise
│               └── ranked_hypotheses
```

Measure dwell time before and after.

That's your before-and-after.

- Wire detector → monitor → agent once
- Scope each tool to one bounded query
- Ship first; measure dwell time as your KPI
- Tune detection before adding more tools

T H A N K Y O U

F R O M

ALERT



T O

ANSWER

- The agent is in the repo.
- The alerting loop is wired up.
- The failure modes are real.
- Questions welcome — especially the ones about production.



From Alert to Answer: Accelerating Anomaly Investigation with
OpenSearch Agent Skills

Resources and contacts

Slides, demos, and additional resources

PRESENTERS

Seema Saharan

Autodesk

[/in/seemasaharan/](#)

Brian Graf

NetApp Instaclustr

[/in/TheBrianGraf](#)

RESOURCE PACK

Slides

PDF

Demo repository

Code, configs, and data

Instaclustr Free Trial

<https://ntap.fyi/free>

ALL LINKS IN ONE PLACE



ntap.fyi/oscna2026